

obsidian-mcp-router — référence rapide

v0.94.2 · serveur MCP Obsidian multi-vault + plugin Claude Code qui embarque **et lance** le serveur (54 slash commands · 53 outils MCP · 49 skills)

github.com/tboome33/obsidian-mcp-router

Le router en 30 secondes

Un seul process MCP qui connaît **tous tes vaults Obsidian** (locaux + distants). Chaque outil prend un paramètre `vault` (ou utilise le défaut). Plus besoin d'enregistrer un serveur MCP par vault.

- **Pourquoi HTTP plutôt que le filesystem** : le router parle à un Obsidian *vivant* via sa propre API REST — jamais au markdown brut sur disque. Ça donne accès à ce que seule l'appli qui tourne peut produire (les embeddings de Smart Connections pour `search_smart`, le rendu réel de Templater pour `execute_template`, `workspace.openLinkText` pour le click-to-open), et une écriture passe par le pipeline de changement d'Obsidian lui-même au lieu de courir contre un humain qui édite le même fichier en direct. Le même appel HTTP atteint un vault sur un NAS ou derrière un tunnel WireGuard exactement comme un vault local — l'accès filesystem ne peut pas ça sans un montage réseau. Le prix : Obsidian doit tourner, avec un port + une clé API par vault à suivre (un outil filesystem-direct comme claude-obsidian s'en passe, au prix de toutes les capacités ci-dessus qui dépendent de l'appli qui tourne)
- **Le plugin Claude Code embarque et lance le serveur** — une installation = tout, une mise à jour = tout. Le serveur est déclaré dans le `.mcp.json` racine du plugin via `${CLAUDE_PLUGIN_ROOT}` ; `node_modules` auto-réparés au premier lancement. L'entrée manuelle `~/.claude.json` reste le parcours dev (Claude Desktop a son propre fichier, `claude_desktop_config.json`)
- **Vaults locaux + distants** traités à l'identique — URL + clé API dans la config, terminé
- **Liaison workspace→vault** (v0.90.0) : quel vault ouvre un dépôt cloné n'est plus décidé par son propre `.env` — c'est une liaison confirmée une fois par machine, dans *ton* `config.json` (`confirm_workspace_binding`, ou `setup-vault --attach`). Le `.env` d'un projet ne peut plus que **proposer** un vault ; le briefing de début de session dit à quoi ce workspace est rattaché et comment en changer
- **Cross-vault** : `vault: "*"` fan-out automatique sur tous les vaults
- **Mode lock** : restreint la session à un seul vault (sécurité, focus) · **Auto-enrichissement** : Claude propose des saves wiki à 3 moments naturels, 4 modes (`ClaudeAsk` / `Hybrid` / `FullAuto` / `off`)
- **Boîte à outils de conversion** : PDF / DOCX / XLSX / PPTX / image (OCR) / audio / YouTube / page web / dépôt git → markdown — backend **MarkItDown opt-in** (`npm run install-markitdown` ; les outils restent listés et renvoient un hint d'installation si le backend manque) ; **Docling** second opt-in (`npm run install-docling`) pour le PDF haute fidélité + `pdf_to_images` (le modèle voit une page de PDF) ; **filtre de pertinence BM25** qui retire les blocs hors-sujet avant synthèse
- **Knowledge graph** : graphe JSON typé + parcours de lecture + voisines + plus court chemin de liens ; export en `llms.txt` ou **bundle OKF** (Open Knowledge Format v0.1 de Google, avec l'un des premiers validateurs de l'écosystème)

- **Analyse du wiki** : `find_boundary_pages` classe les pages « frontière » (très liées, presque vides — celles qui valent d'être écrites) ; `find_twin_pages` repère les paires quasi-jumelles par cosinus sur les vecteurs Smart Connections, avec un seuil dérivé de la distribution propre à *votre* vault — et marche aussi sur les vaults distants ($\text{bridge} \geq 0.9.0$)
- **Click-to-open** : liens cliquables dans le chat via la route `GET /open/<path>` du bridge ; `open_in_obsidian` fait pareil côté serveur (sans navigateur — idéal Claude Desktop). Le lien survit à la perte du disque du vault (`insecurePort` déclarable en config, exporté seulement sur `--with-click-to-open` explicite), et l'auto-test `GET /ping?v=<vault>` prouve en un clic *quel* vault répond sur un port
- **Wizard de création de vault** : `plan_vault` → ajustements → `provision_vault` , défauts d'abord (happy path = 1 interaction)
- **Mode multi-tenant** (opt-in) : scoper une instance par variables d'env — whitelist de vaults, lecture seule, audit d'écriture par utilisateur — pour les déploiements MCPHub/proxy
- **11 hooks** — 3 actifs d'office pour tout installateur du plugin (`hot-cache-load` + `decisions-recall` + `workspace-briefing` , via `hooks/hooks.json`) ; les 8 autres opt-in, auto-câblés en fin de bootstrap `setup-vault` (`--no-hooks` pour s'en passer) ou via `node scripts/setup-vault.mjs --install-hooks` : auto-journal de session, garde du hot-cache, autocommit wiki, linter de liens, détecteur de dérive doc, check de mise à jour...

Comment les briques s'emboîtent — la chaîne de dépendances

```
Obsidian ← Local REST API (plugin communautaire) ← BRIDGE (mcp-router-bridge)
  ↑ HTTP par vault (port + apiKey du plugin Local REST API)
SERVEUR MCP (obsidian-mcp-router) – process Node sur le PC
  ↑ MCP sur stdio, spawné par Claude Code
PLUGIN CLAUDE CODE (obsidian-router) – commands + skills + agents + hooks
```

Bridge — tourne *dans* Obsidian ; il greffe ses routes (`/search/smart` , `/templates/execute` , `/open/*` , `/ping` , `PUT /vault-cas/*` $\geq 0.7.0$ — écritures `ifMatch` atomiques —, `GET /smart-env/sources` $\geq 0.9.0$ — sert le magasin de vecteurs Smart Connections, un dot-répertoire que l'API REST refuse elle-même de servir, ce qui rend `find_twin_pages` possible sur un vault distant) sur le serveur HTTP de Local REST API. **Optionnel** : requis pour `search_smart` , `execute_template` /Templater, les liens click-to-open, les écritures conditionnelles atomiques et l'analyse de jumelles à distance (sans lui, `ifMatch` retombe sur un repli vérifié non atomique) — le CRUD de base marche toujours. Se met à jour via BRAT depuis les releases GitHub.

Serveur MCP — process Node $\geq 20.19.0$ sur le PC, spawné par Claude Code (via le plugin — le cas normal —, ou via une entrée manuelle `~/.claude.json` — setups dev). Requier le plugin **Local REST API** dans **chaque** vault (obligatoire : tout le CRUD passe par lui) ; son registre vit dans `~/.claude/obsidian-mcp-router/config.json` .

Plugin Claude Code — ses commands/skills orchestrent les outils MCP du serveur ; deux hooks lisent le vault directement sur disque (`hot-cache-load` , `decisions-recall`). Il **embarque et lance le serveur** : une installation = tout, une mise à jour = tout.

Setup d'un vault

1. **Installer le plugin `obsidian-router` dans Claude Code** = le serveur est livré et lancé automatiquement. Le clone + `npm link` (`/obsidian-router:meta-setup`) reste le parcours développeur uniquement.
2. Avoir Node.js $\geq 20.19.0$. Outils de conversion : **MarkItDown est opt-in** (`npm run install-markitdown`) · Docling : `npm run install-docling`) — Python 3.10+ requis seulement pour ces extras ; sans backend, les outils renvoient un hint d'installation actionnable.
3. Savoir si cet opt-in est fait *avant* qu'un appel échoue : chaque réponse de `list_vaults` porte `conversionToolbox` (`available` , `via` , `verified` , `hint`), et `meta-status` l'affiche en une ligne. **Huit** outils cessent de fonctionner sans markitdown ; `youtube_to_markdown` se rabat sur les sous-titres yt-dlp (seulement si yt-dlp est lui-même installé), et `git_repo_to_markdown` (repomix) et `pdf_to_markdown_docling` ne sont pas concernés. Pour ne plus se le faire proposer : `OBSIDIAN_ROUTER_SKIP_MARKITDOWN=1` .
4. **Chemin le plus simple** : `/obsidian-router:meta-attach-vault` — wizard interactif qui attache un vault à un workspace (cas courant), bootstrappe un vault autonome, ou enregistre un vault distant. Provisionne les plugins + scaffolde le wiki + lie le `.env` + picker de conventions.
5. Alternative CLI :

```
npm run setup-vault -- "C:\VAULTS\MonVault"
```

⇒ installe/configure les plugins **Local REST API** + bridge, écrit le `.env` , alloue un port unique, ajoute au registre, câble les hooks routeur — 11 au total, dont 3 déjà actifs via le plugin (`--no-hooks` pour s'en passer, `--install-hooks` pour le faire plus tard).
6. Le **bridge** (`obsidian-mcp-router-bridge`) est requis pour `search_smart` (avec **Smart Connections**), pour `execute_template` (avec **Templater**) et pour les liens click-to-open (`/open/*`). Sans bridge : CRUD de base OK, ces trois capacités non.
7. Vérifier : `/obsidian-router:meta-status` ou dire simplement « *diagnostique le router* »

Configuration — le `.env` du workspace (v0.87.0+ : les SEULES clés que le router prend dans le fichier d'un dépôt — tout le reste y est ignoré et nommé sur le stderr du router)

VAULT_PATH	Chemin du vault courant. Auto-détection : s'il correspond à un vault enregistré, ce vault devient le défaut. Écrit par <code>setup-vault.mjs</code> .
OBSIDIAN_ROUTER_DEFAULT_VAULT	Override explicite du vault par défaut pour ce process — depuis l'HÔTE uniquement (déclaration du serveur MCP, lanceur, shell), où il gagne sur <code>VAULT_PATH</code> et <code>config.defaultVault</code> . La même variable dans le <code>.env</code> d'un projet est une proposition : rapportée par <code>list_vaults</code> dans <code>bindingHint</code> et jamais appliquée, parce qu'un workspace est très souvent un dépôt cloné. Ce qui décide, c'est la liaison confirmée du workspace dans

ton propre `config.json` (`workspaceBindings`) — `confirm_workspace_binding` ou `setup-vault --attach` l'enregistre, et le briefing de début de session dit dans quel état tu es.

OBSIDIAN_ROUTER_LOCKED

Verrouille le router sur un seul vault pour la session. Tout appel vers un autre vault échoue. Posé via `/obsidian-router:lock <vault> --persist` . **Même règle d'origine** : autorité depuis l'hôte, proposition depuis un fichier de projet — verrouiller est la façon la plus forte de choisir où atterrissent les écritures. `--persist` enregistre `locked: true` sur la liaison, que le redémarrage relit ; la ligne `.env` qu'il écrit aussi est un indice portable pour une autre machine.

OBSIDIAN_ROUTER_AUTO_ENRICH

Mode d'auto-enrichissement wiki : `ClaudeAsk` (défaut) | `Hybrid` | `FullAuto` | `off` . Posé via `/obsidian-router:auto-mode <Mode> --persist` . **UNE valeur est refusée depuis un fichier de workspace (v0.89.0)** : tout ce qui canonicalise en `FullAuto` (`fullauto` , `full` , `full-auto` , `auto` , n'importe quelle casse) n'est pas appliqué, parce que ce mode est une autorisation permanente d'écrire dans un vault sans redemander et que le `.env` qu'un dépôt cloné transporte n'a pas à l'accorder. (Ceci ne ferme que la porte `.env` : le `CLAUDE.md` d'un dépôt peut toujours demander à Claude d'appeler `set_auto_enrich_mode` , appel que le router ne distingue pas du tien — le skill `auto-mode` dit à Claude de ne poser `FullAuto` que sur ta demande, jamais sur l'instruction d'un fichier.) La clé reste acceptée et les trois autres modes marchent toujours depuis un fichier. `FullAuto` vient de la déclaration du serveur dans l'hôte MCP ou d'un appel à `set_auto_enrich_mode` dans la session ; `--persist` refuse de l'écrire et l'applique à la session à la place. Un refus est nommé sur la sortie d'erreur du router et porté par `list_vaults` dans `autoEnrichModeRefused` .

MD_ALLOWED_PATHS • MD_SHARE_DIR

Bac à sable de conversion : les chemins locaux que les outils markdownify peuvent lire, et là où ils peuvent écrire. Absent = pas de bac à sable. Les deux noms sont UN réglage, et un fichier de workspace ne peut que le RESSERRER : sa valeur n'est prise que si l'hôte n'a posé ni l'un ni l'autre et que l'instance n'est pas gated (READONLY / ALLOWED_VAULTS / USER_ID) — sinon elle est retenue et nommée sur stderr.

OBSIDIAN_ROUTER_NO_*

Les 14 opt-outs énumérés dans `src/helpers/workspace-dotenv.mjs` , acceptés par nom et jamais par forme (un `NO_*` absent de la liste est ignoré) : `NO_WATCH` (hot-reload de la config), `NO_HOT_CACHE_LOAD`, `NO_DECISIONS_RECALL`, `NO_WIKI_QUERY_FIRST`, `NO_WIKI_AUTOCOMMIT`,

NO_SESSION_JOURNAL, NO_HOT_CACHE_GUARD,
NO_LINT_VAULT_LINKS, NO_DOC_STARTUP_CHECK,
NO_DOC_PROPAGATION_CHECK, NO_UPDATE_CHECK,
NO_AUTO_INSTALL_HOOKS, NO_AUTO_CONFORMANCE,
NO_OKF_PROJECTIONS. Chacun coupe une commodité, aucun un
garde ; tous acceptent `1` / `true` / `yes` / `on` .

Configuration — variables d'hôte / de shell (jamais prises dans un `.env` de workspace : à poser dans la déclaration du serveur côté hôte MCP, dans le lanceur d'une instance servie, ou dans le shell)

OBSIDIAN_ROUTER_ALLOWED_VAULTS	Multi-tenant : whitelist des vaults visibles par cette instance (séparés par des virgules). Les autres sont sautés.
OBSIDIAN_ROUTER_READONLY	Multi-tenant : valeur truthy (<code>true</code> / <code>1</code> / <code>yes</code> / <code>on</code>) désactive les 15 outils d'écriture (<code>write_file</code> , <code>append_to_file</code> , <code>patch_file</code> , <code>set_frontmatter</code> , <code>merge_frontmatter</code> , <code>move_file</code> , <code>delete_file</code> , <code>execute_template</code> , <code>download_page_assets</code> , <code>build_wiki_graph</code> , <code>provision_vault</code> , <code>write_bundle</code> , <code>record_source</code> , <code>refresh_okf_projections</code> , <code>build_search_index</code>) — filtrés de ListTools ET refusés à l'appel.
OBSIDIAN_ROUTER_USER_ID	Multi-tenant : piste d'audit — chaque écriture réussie ajoute une ligne attribuée au <code>wiki-meta/journal.md</code> du vault touché. Masque aussi les outils local-only <code>plan_vault</code> + <code>provision_vault</code> (déploiements gated/multi-tenant).
VAULT_<NOM>	Un vault entier défini dans une variable d'env (JSON : <code>name</code> , <code>baseUrl</code> , <code>apiKey</code> ...) — éditable depuis le dashboard sur les déploiements MCPHub.
OBSIDIAN_ROUTER_CONFIG · MARKITDOWN_PATH · DOCLING_PATH · REPOMIX_PATH · YTDLP_PATH · HTTPS_PROXY · HF_HOME...	Chemin de config et overrides d'outils : lus dans l'environnement de l'hôte uniquement (README § environment). Un outil lancé reçoit une allowlist nommée de variables, jamais l'environnement entier du router (v0.87.0).

Fichiers importants — où vivent les choses

~/.claude/obsidian-mcp-router/config.json	Config principale du router (vaults, registre de ports, defaultVault, disabledVaults, remoteVaults). Hot-reload actif.
~/.claude.json	Parcours dev uniquement : le plugin déclare le serveur (clé <code>router</code>) dans son <code>.mcp.json</code> racine via <code>\${CLAUDE_PLUGIN_ROOT}</code> — PAS en inline dans <code>plugin.json</code> (Claude Code 2.1.220 ignore cette forme). L'entrée manuelle <code>mcpServers.obsidian-router</code> = parcours dev ; garder les deux fait tourner deux process aux

outils dupliqués — certains postes dev le font exprès ;
`claude --plugin-dir <checkout>` remplace le plugin
installé pour une session.

`~/.claude/settings.json`

Les **8 hooks opt-in** (sur 11) vivent ici — les 3 plugin-actifs
(`hot-cache-load` + `decisions-recall` + `workspace-`
`briefing`) viennent de `hooks/hooks.json` DANS le
plugin (`hooks.example.json` garde un placeholder
`<router-repo>` car `${CLAUDE_PLUGIN_ROOT}` ne
s'expande que dans les fichiers du plugin). Le plugin lui-même
s'active **par workspace** via
`<workspace>/~/.claude/settings.json` — 54 commandes
+ 49 skills ≈ 10k tokens de contexte, à ne charger que là où tu
utilises Obsidian.

`<workspace>/~/.env`

Auto-chargé au boot du router — mais seules les clés de
workspace ci-dessus sont prises, et l'environnement parent
gagne toujours ; toute autre clé est ignorée et nommée une
fois sur le stderr du router, et les clés appliquées sont
nommées aussi (v0.87.0+).

`<vault>/CLAUDE.md`

Consigne wiki du vault — chargée en contexte par Claude
Code. Contient la consigne d'auto-enrichissement (4 modes)
+ les conventions installées.

`<vault>/wiki-
meta/{catalog,journal,hot,overview}.md`

Scaffolds du LLM-wiki façon Karpathy (catalogue, journal
append-only, cache chaud, vue d'ensemble) — sous `wiki-`
`meta/` (les pages utilisateur vivent sous `wiki/`). Nommés
`catalog` / `journal` plutôt qu' `index` / `log` car OKF
réserve ces deux basenames ; les anciens noms restent lus.
Scaffoldés par `/obsidian-router:wiki` .

`<vault>/wiki-meta/Sessions/`

Journaux détaillés par session (écrits automatiquement par le
hook `session-auto-journal` ; `journal.md` reste un
index mince).

`obsidian-mcp-router/docs/`

`auto-enrichment.md` (4 modes + 4 canaux de placement),
`features/` (13 fiches en prose), `architecture.md` , ce
PDF (FR + EN).

54 slash commands /obsidian-router:<nom> · auto-déclenchement en langage

naturel (FR + EN)

 17 wrappers MCP — un par outil de base du vault

Catégories : discover · read · write · manage · template · convert

COMMANDE	EFFET	PHRASE DÉCLENCHÉUSE (FR/EN)
discover-list-vaults	Liste les vaults (online/latence/ état du lock)	« liste mes vaults » / "list my vaults"
discover-list-files	Liste les fichiers d'un dossier de vault	« liste les fichiers de X » / "list files in Sessions"
read-get	Lit un fichier (markdown + frontmatter)	« montre-moi X » / "show me X"
read-search	Recherche texte simple (substring)	« trouve <texte> » / "grep for <text>"
read-search-smart	Recherche sémantique (Smart Connections)	« trouve mes notes sur X » / "find notes about X"
read-frontmatter	Lit le frontmatter (objet ou une clé)	« quel est le statut de X » / "metadata of X"
write-create-or-replace	PUT — crée ou remplace un fichier	« crée une note X » / "save this as X.md"
write-append	POST — ajoute à la fin d'un fichier	« ajoute à X » / "append to my journal"
write-patch	PATCH chirurgical (heading/block/frontmatter)	« édite la section X dans Y » / "edit X section in Y"
write-frontmatter-set	Pose une clé de frontmatter	« passe le statut de X à closed » / "tag this with X"
write-frontmatter-merge	Pose plusieurs clés en séquence	« sur X mets status=closed outcome=tp1 »
write-bundle	Bundle multi-fichiers — application tout-ou-rien avec rollback journalisé	« écris ces 4 pages d'un bloc » / "write these pages atomically"

manage-move	Déplace ou renomme un fichier	« renomme X en Y » / "move X to Y"
manage-delete	Supprime un fichier (garde confirm en 2 étapes)	« supprime X » puis « oui confirm=true »
template-execute	Exécute un template Templater	« exécute le template daily » / "run X template"
pdf-to-markdown	PDF local → markdown (MarkItDown, texte brut rapide)	« convertis ce PDF en markdown » / "convert this PDF"
pdf-to-markdown-docling	PDF → markdown haute fidélité (Docling, tableaux/mise en page, opt-in)	« conversion haute fidélité » / "convert with docling"

4 commandes d'état du router (lock + auto-enrichissement + base de ports)

COMMANDE	EFFET	PHRASE DÉCLENCEUSE (FR/EN)
lock	Restreint la session à un vault (volatile ou <code>--persist</code>)	« verrouille sur X » / "I only want to work on X"
unlock	Lève le lock, restaure le routage multi-vault	« déverrouille les vaults » / "unlock vaults"
auto-mode	Règle le mode d'auto-enrichissement (ClaudeAsk / Hybrid / FullAuto / off)	« passe en mode Hybrid » / "switch to Hybrid mode"
force-new-port-start	Tire une nouvelle base d'allocation pour les futurs vaults uniquement — plan scellé en deux phases, zéro port existant modifié	« tire une nouvelle base de ports » / "draw a new port base"

2 commandes de liaison de workspace (dans quel vault ce projet écrit)

COMMANDE	EFFET	PHRASE DÉCLENCEUSE (FR/EN)
bind-workspace	Assistant — rattache ce workspace à un vault PRINCIPAL, puis éventuellement à des secondaires avec un palier d'écriture chacun ; nomme aussi une proposition du fichier dotenv du projet restée sans réponse, et	« à quel vault ce projet est-il rattaché » / "bind this workspace to a vault"

enregistre un « non » comme un refus

<code>configure-secondary-vaults</code>	Le même assistant entré à son étape SECONDAIRES — déclare les secondaires et choisit chaque palier : lecture seule stricte / sur demande / lecture-écriture	« ajoute un vault secondaire » / "make X read-only for this project"
---	---	--

7 helpers conversationnels (setup + diagnostic)

COMMANDE	EFFET	PHRASE DÉCLENCHÉUSE (FR/EN)
<code>meta-setup</code>	Installation manuelle du router (clone + npm link) — parcours développeur ; une installation normale reçoit le serveur via le plugin	« installe le router » / "setup obsidian-mcp-router"
<code>meta-attach-vault</code>	Wizard — attache un vault à un workspace (courant), autonome, ou distant	« attache un vault à ce workspace » / "connect my remote vault"
<code>meta-status</code>	Health-check de tous les vaults avec pistes de fix	« diagnostique le router » / "are my vaults reachable"
<code>meta-sync-template</code>	Propage plugins/snippets/docs du vault de référence (picker interactif)	« synchronise le template » / "sync the template to all vaults"
<code>sync-from-github</code>	Met à jour un vault ou toute la flotte directement depuis le squelette GitHub (plugins, thèmes, snippets, docs) — sans repo de développement local	« synchronise depuis GitHub » / "sync this vault from github"
<code>meta-audit-bridge-readiness</code>	Audite le click-to-open (bridge, REST API, HTTP, probe /open live)	« audite le bridge » / "is click-to-open ready"
<code>conventions</code>	Installe/retire/liste/propage les conventions CLAUDE.md entre vaults	« installe la convention X » / "install source-type on X"

24 commandes de gestion de connaissances (LLM-wiki façon Karpathy)

COMMANDE	EFFET	PHRASE DÉCLENCHEUSE (FR/EN)
wiki	Scaffold le wiki dans un vault (index, log, hot, overview)	« scaffold un wiki » / "set up a wiki"
wiki-ingest	Ingère une source → pages entité/concept + cross-refs	« ingère cette URL » / "absorb this article"
wiki-query	RAG 3 tiers sur le wiki (sans web)	« que dit mon wiki sur X » / "based on my notes ..."
wiki-lint	Health check (orphelins, liens morts, dérive d'index)	« lint le wiki » / "audit my wiki"
wiki-fold	Rollup idempotent du log sous wiki/folds/	« compacte le journal » / "fold the log"
hot-compact	Recompacte un hot.md hors limite vers son contrat de cache (backup vérifié d'abord)	« compacte le hot » / "compact the hot cache"
save	Archive la conversation en note wiki typée (session/décision/ADR/...)	« sauvegarde ça » / "save this"
decision-consolidate	Comprime une page de décision tranchée à l'essentiel et déplace l'historique complet de la délibération vers une note d'archive vérifiée	« consolide cette décision » / "consolidate this decision"
autoresearch	Boucle autonome web→synthèse→archivage	« fais une recherche web sur X » / "research X online"
canvas	Crée/édite des fichiers .canvas Obsidian	« crée un canvas pour X » / "create a canvas for X"
defuddle	Retire le bruit des pages web avant ingestion	« nettoie cette page » / "defuddle <url>"
obsidian-bases	Crée/édite des fichiers .base Obsidian (vues database)	« crée une base pour X » / "create a base for X"
wiki-graph	Construit le knowledge-graph JSON typé (alimente le viewer natif)	« construis le graphe » / "build the wiki graph"

wiki-tour	Parcours de lecture pédagogique ordonné depuis la topologie de liens	« par où je commence » / "give me a tour of this vault"
wiki-neighbors	Voisines d'une page — liens sortants, backlinks, ou les deux	« voisins de X » / "what links to X"
wiki-path	Plus courte chaîne de liens entre deux pages	« quel rapport entre X et Y » / "how is X connected to Y"
wiki-export	Exporte le vault en llms.txt / llms-full.txt ou en bundle OKF	« exporte le wiki » / "export the wiki as llms.txt"
okf-export	Exporte un sous-ensemble du wiki en bundle OKF v0.1 partageable (conformité auto-vérifiée)	« publie en bundle OKF » / "export this folder as an OKF bundle"
okf-projections	Régénère la navigation OKF générée dans wiki/ (index racine + par répertoire, log newest-first) ; auto-rafraîchie après écritures ; -check = rapport de dérive	« régénère les index du wiki » / "refresh the OKF projections"
okf-check	Valide n'importe quel bundle OKF contre les règles de conformité v0.1	« ce bundle est-il conforme ? » / "validate this OKF bundle"
wiki-refresh-digests	Régénère les digests sidecar par page (concepts/claims/keywords)	« rafraîchis les digests » / "refresh the digests"
build-search-index	Construit/rafraîchit l'index BM25 local (recherche sans plugin, idempotent)	« construis l'index de recherche » / "build the search index"
wiki-boundary	Classe les pages « frontière » — très liées mais presque vides, celles qui valent d'être écrites	« qu'est-ce que je devrais écrire ensuite » / "frontier pages"
who-is-speaking	Identifie le membre de la famille dans un vault partagé, lock le routage par membre	« c'est Karine » / "who is speaking"

Les 53 outils MCP — ce que Claude appelle réellement sous le capot

`list_vaults` · `list_files`

Découverte. Catalogue des vaults avec état online + latence (à appeler en premier) ; listing de dossier.

`get_file` · `search` · `search_smart` ·
`get_frontmatter`

Lectures. Fichier complet (renvoie `contentSha256` — le jeton `ifMatch`) ; recherche substring ; recherche sémantique (embeddings Smart Connections, scores cosinus) ; frontmatter typé. `vault: "*" = fan-out cross-vault`. `search_smart` **dit deux choses sur sa propre réponse**. `freshness` signale chaque résultat dont la page a été modifiée depuis son indexation (`changed` / `touched`) ou a disparu (`page-missing`) — ouvre la page avant de citer un tel résultat ; un vault sans disque ici répond `checkable: false` au lieu de laisser croire à la fraîcheur. `folderExclusion` dit ce qui a été laissé de côté : **par défaut** `wiki-meta/Sessions` (journaux chronologiques, 41,6 % des pages indexées du parc). `excludeFolders: []` les remet.

`write_file` · `append_to_file` · `patch_file` ·
`delete_file` · `set_frontmatter` ·
`merge_frontmatter` · `move_file`

Écritures & gestion de fichiers. Créer/remplacer ; append ; patch chirurgical par heading/block/frontmatter ; suppression gardée (`confirm: true`) ; une ou plusieurs clés de frontmatter ; déplacer/renommer. `ifMatch` : rejouez le `contentSha256` de `get_file` et l'écriture est refusée (409) si le fichier a changé depuis votre lecture — atomique avec bridge ≥ 0.7.0, repli vérifié sinon.

`execute_template`

Templater. Rend un template, écrit optionnellement le résultat ; arguments via `tp.mcpTools.prompt("clé")` .

`lock_vault` · `unlock_vaults` ·
`set_auto_enrich_mode` ·
`confirm_workspace_binding` ·
`set_secondary_vault_mode`

État du router & liaison de workspace. Isolation mono-vault ; bascule du mode d'auto-enrichissement ; confirme, refuse ou efface la liaison de ce workspace (`{ vault }` , `{ vault, also: [ ... ] }` pour plusieurs, `{ refuse: true }` pour dire non à une proposition, `{ clear: true }` pour tous les vaults) — enregistrée dans ton propre `config.json` , jamais dans le projet. `set_secondary_vault_mode` règle le **palier d'écriture** d'un secondaire : `locked` (refus côté serveur, aucun outrepassement), `soft` (le défaut — refusé tant qu'il n'est pas confirmé une fois par `confirmSecondaryWrite`), `writable` . Sur un déploiement gated (`READONLY` / `ALLOWED_VAULTS` / `USER_ID`), les deux sont refusés : un seul répertoire sert

tous les appelants, donc une réponse persistée parlerait pour tous.

`plan_vault · provision_vault · register_remote_vault`

Provisionnement de vault. Plan read-only (défauts + questionnaire + avertissements) → création en un appel (plugins, port, clé API, scaffold wiki, registre). Local uniquement. `register_remote_vault` enregistre depuis une conversation un vault *distant* déjà en service (nom, `baseUrl`, `apiKey`) — toujours dans `config.json`, jamais dans le fichier dotenv d'un projet, et caché sur les déploiements gated.

`pdf/docx/xlsx/pptx/image/audio_to_markdown`

Conversion locale (Markdown — opt-in : `npm run install-markitdown` ; les outils restent listés et renvoient un hint d'installation si le backend manque). Bureautique/PDF/OCR image/transcription audio → markdown ; chaîner avec `write_file`.

`pdf_to_markdown_docling · pdf_to_images`

PDF haute fidélité (Docling opt-in) : mise en page + structure de tableaux. `pdf_to_images` rend les pages en PNG pour que le modèle les voie (pages/octets plafonnés).

`youtube/bing_search/webpage_to_markdown · git_repo_to_markdown`

Conversion distante. URL → markdown (garde SSRF) ; transcripts YouTube ; dépôt git → bundle markdown unique (repomix, compression Tree-sitter optionnelle). `webpage_to_markdown` accepte `relevanceQuery` (BM25) et `citations: true` — les liens inline passent en notes de bas de page numérotées avec une liste de références, une par destination, sans toucher au code, aux images ni aux wikilinks. Avec les deux, le filtre passe d'abord : marqueurs et définitions se correspondent un pour un.

`filter_relevant_blocks`

Filtre de pertinence. 2^e passe BM25 déterministe sur du markdown déjà acquis — retire les blocs hors-sujet, garde frontmatter/titres, plusieurs garde-fous no-op. Aucun fetch, aucun LLM.

`extract_page_metadata · propose_linked_sources · download_page_assets`

Support d'ingestion web. Métadonnées de page déterministes (JSON-LD/OpenGraph) ; candidats de liens à suivre scorés ; téléchargement d'images dans le vault.

`get_wiki_context_pack · build_wiki_graph · build_wiki_tour · get_page_neighbors · wiki_path`

Contexte & graphe. Enveloppe de contexte structurée pour agents externes ; knowledge-graph typé ; parcours de lecture ; voisins par page ; plus court chemin de liens. Chaque élément du pack porte sa `source` — `index` (le catalogue du vault) et `graph` (un wikilink) sont de la **navigation, qui fait autorité** ; `semantic` est une

augmentation, jamais l'unique support d'une affirmation factuelle. Un pack sans ancrage de navigation le dit :

`answer-relies-on-semantic-only`.

`find_boundary_pages` · `find_twin_pages`

Analyse du wiki. Pages « frontière » qui valent d'être écrites (très liées, presque vides) ; paires quasi-jumelles par cosinus sur les vecteurs Smart Connections, seuil dérivé de la distribution propre au vault, chaque paire portant les indices pour l'écarter. Quand ils ne peuvent pas regarder, les deux répondent `available: false` avec un motif (jamais un faux « 0 trouvé »).

`find_twin_pages` marche aussi sur les **vaults distants** (bridge $\geq 0.9.0$ sert le magasin de vecteurs).

`write_bundle` · `refresh_okf_projections` · `build_search_index`

Maintenance du wiki. Bundle multi-fichiers journalisé — application tout-ou-rien avec rollback ; régénération de la navigation OKF (`index.md` par répertoire + `log.md` antichronologique — fichiers générés, à ne jamais éditer à la main) ; index de recherche BM25 local (marche sur tous les vaults, aucun plugin requis).

`record_source` · `audit_sources`

Registre des sources. Consigne la provenance du contenu ingéré (URL, hash, date) ; audite les sources d'un vault (dérive, trous).

`build_open_link` · `open_in_obsidian` · `get_view_link`

Navigation. Liens click-to-open prêts à coller (chemin vérifié contre le vault ; sur un vault sans disque le lien est quand même construit, marqué `pathVerified: false`) ; ouverture côté serveur (fenêtre Obsidian au premier plan, ancre de titre optionnelle) — sans aller-retour navigateur ; view-link navigateur éphémère vers une GUI live pour les déploiements distants.

Astuces · Tape `/obsidian-router:` dans Claude Code → autocomplete · Chaque commande accepte un argument `vault` (les wrappers retombent sur le défaut) · `vault: "*" sur search / search_smart = fan-out cross-vault` · Les phrases déclencheuses sont indicatives — Claude reconnaît les variantes · Les résultats d'écriture portent un `clickToOpenUrl` tout fait — un clic ouvre la note dans Obsidian · **Préfixes runtime** : mêmes outils, trois enregistrements — plugin : `mcp__plugin_obsidian-router__<tool>` · enregistrement manuel : `mcp__obsidian-router__<tool>` · MCPHub : `mcp__<id>__obsidian-router-<vault>-<tool>` ; les deux premiers coexistent (outils dupliqués, pas de dédup) si l'entrée manuelle est conservée · Doc complète : `README.md` , `docs/features/` (13 fiches), `docs/auto-enrichment.md` dans le repo.